

Uniwersytet Rzeszowski

Wydział Nauk Ścisłych i Technicznych



Jakub Płocica 1234960

Kacper Ręczak 134968

Kamil Szopniewski 134976

EnemyAI – Inteligentny system sterowania statkiem kosmicznym w przestrzeni 3D

Dokumentacja projektowa

Projekt zaliczeniowy przedmiotu Sztuczna inteligencja

Prowadzący:

mgr inż. Marcin Mrukowicz

Rzeszów, 2026 r.

Spis treści

1	Wstęp	2
1.1	Cel projektu	2
2	Teoria i implementacja wykorzystanych algorytmów sztucznej inteligencji	3
2.1	Algorytm A* w dynamicznej przestrzeni 3D	3
2.2	Algorytm Minimax z odcięciem $\alpha\text{-}\beta$	3
3	Przebieg rozgrywki pomiędzy człowiekiem a sztuczną inteligencją	5
4	Napotkane problemy, wyzwania i sposoby ich rozwiązania	7
5	Wykorzystanie Generatywnej Sztucznej Inteligencji	8
6	Podsumowanie i wnioski z implementacji AI	9

1. Wstęp

1.1. Cel projektu

Projekt dotyczy zaimplementowania inteligentnego systemu sztucznej inteligencji (AI) przeciwnika w grze symulującej walkę statków kosmicznych w przestrzeni trójwymiarowej (przestrzeń kosmiczna z pasem asteroid), wykorzystującej silnik Unity 6 i fizykę Rigidbody. Zgodnie z głównymi założeniami projektowymi przedmiotu Sztuczna Inteligencja, celem było stworzenie zaawansowanego statku AI działającego w modelu **Predictive-Reactive**. System rozwiązuje praktyczny problem nawigacji w środowisku o zmiennej strukturze grafu oraz podejmowania decyzji taktycznych w warunkach niepewności, symulując walkę burtową okrętów.

Statek AI operuje wyłącznie w oparciu o zaimplementowane od podstaw zaawansowane algorytmy decyzyjne: algorytm Minimax z odcięciem alfa-beta oraz algorytm heurystycznego przeszukiwania A^* w pełnej przestrzeni 3D.

2. Teoria i implementacja wykorzystanych algorytmów sztucznej inteligencji

2.1. Algorytm A* w dynamicznej przestrzeni 3D

Podstawy teoretyczne: Algorytm A* to heurystyczny algorytm przeszukiwania grafu, gwarantujący znalezienie najkrótszej ścieżki (o ile użyta heurystyka jest dopuszczalna). W klasycznym ujęciu oblicza on funkcję kosztu $f(n) = g(n) + h(n)$, gdzie $g(n)$ to rzeczywisty koszt dotarcia ze startu do węzła n , a $h(n)$ to szacowany (heurystyczny) koszt dotarcia z węzła n do celu. Ze względu na w pełni zaimplementowane środowisko przestrzeni kosmicznej, problemem staje się nawigacja w sześciu stopniach swobody, co wyklucza użycie płaskich struktur grafowych.

Szczegóły implementacji w kodzie: Odgórny wymóg manualnego odnajdywania optymalnej ścieżki wykluczył użycie natywnego rozwiązania NavMeshAgent. Główna implementacja nawigacji opiera się na skrypcie `ManualAStar.cs`. Mapa przed rozpoczęciem rozgrywki dzielona jest na sześciany w ramach trójwymiarowej siatki wokseli (Voxel Grid), reprezentowanych strukturą `Node[x, y, z]`. Algorytm korzysta z autorskiej, zoptymalizowanej kolejki priorytetowej w celu dobierania węzła o najniższym współczynniku $f(n)$.

W funkcji `CalculatePath()` silnik wylicza dystans heurystyczny przy użyciu klasycznej miary euklidesowej: $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$. Z uwagi na zmienność środowiska, podczas przeszukiwania węzły weryfikują zajętość miejsca (skanowane asteroidy) za pomocą rzutowania `Physics.OverlapBox`. Z powodu znacznej rozpiętości siatki Grid 3D obliczenia powodowały znaczne spadki wydajności i utratę płynności renderowania obrazu. Zastosowano zaawansowane rozwiązanie optymalizacyjne: algorytm został zaimplementowany jako współprogram (ang. *Coroutine*) przy użyciu interfejsu `IEnumerator`. Główna pętla `while` wykonująca ekspansję węzłów inkrementuje licznik zbadanych elementów. Po osiągnięciu stałej wartości (40 operacji), skrypt wywołuje instrukcję `yield return null`, co powoduje zawieszenie działania algorytmu A* do kolejnej klatki renderowania. Podejście to gwarantuje zachowanie płynności na poziomie 60 FPS, unikając jednoczesnego zablokowania wątku głównego (ang. *main thread blocking*) i nie powodując odczuwalnych opóźnień w logice decyzyjnej przeciwnika.

2.2. Algorytm Minimax z odcięciem α - β

Podstawy teoretyczne: Minimax to klasyczny algorytm z zakresu sztucznej inteligencji oraz teorii gier o sumie zerowej z pełną informacją. Wybiera on akcję, która minimalizuje maksymalną możliwą stratę – matematycznie zabezpiecza pozycję, zakładając, że oponent zawsze odpowie dla niego najlepszym z możliwych ruchów. Z uwagi na wykładniczą złożoność na drzewach decyzji (drzewo manewrów w przestrzeni gier dynamicznych), wprowadzono al-

gorytm odcięcia alfa-beta. Służy on do zmniejszenia liczby węzłów ocenianych w drzewie poprzez zaniechanie eksploracji gałęzi, jeżeli nie przyniosą one wyższej wygranej niż ścieżki już odkryte.

Szczegóły implementacji w kodzie: Podczas gdy statek przechodzi w stan "Combat", moduł sztucznej inteligencji zaczyna stale wywoływać rekurencyjną metodę w języku C#, posiadającą sygnaturę bliską `float Minimax(State node, int depth, float alpha, float beta, bool isMaximizingPlayer)`. Algorytm generuje hipotetyczne pozycje własnego statku (np. rotacja $+25^\circ$, lot w przód, ucieczka) względem przewidywanych wektorów prędkości ucieczki gracza w perspektywie najbliższych dwóch sekund gry.

Najbardziej kluczowym elementem implementacji logicznej jest matematyczna funkcja ewaluacyjna wywoływana dla każdego liścia decyzyjnego (`EvaluateState()`). Oblicza ona wynik taktyczny punktując 3 główne wagi:

- **Kąt ostrzału i manewr burtowy:** Wykorzystując mechanikę wektorową silnika Unity (`Vector3.Dot()`), sprawdzane jest odchylenie katowe pomiędzy zwrotem gracza a statkiem AI. Silnik punktuje ekstremalnie wysoko pozycje dążące do zachowania kąta 90 stopni (burta wystawiona względem przodu gracza). Maksymalizuje to pole rażenia własnych artylerii, dając przewagę w ostrzale.
- **Zasięg optymalny (Dystans):** Obliczana znormalizowana wartość preferuje dystans od 80 do 120 jednostek. Przekroczenie go zmniejsza punkty (groźba ucieczki), a zbytne skrócenie go skutkuje nałożeniem skrajnie negatywnych wag liczbowych (kary punktowe), co zmusza system decyzyjny AI do unikania zderzeń czołowych i taranowania gracza.
- **Punkty vitalne i pancierz strefowy:** System ma dostęp do referencji na `ShipStats.cs`. W funkcji heurystycznej wynik jest znacząco redukowany, jeżeli sprawdzany kierunek lotu wystawia na cios strefę pancierza (np. uszkodzony tył), która posiada krytycznie niski poziom punktów wytrzymałości.

Algorytm alfa-beta z sukcesem odrzuca liczne gałęzie drzewa symulacji, w których straty dla statku są bezcelowo głębokie. Finalnie wybrany ruch optymalny jest tłumaczony na wektory pędu w C# i przekazywany bezpośrednio do modułu lotu opartego o wektory `Rigidbody`.

3. Przebieg rozgrywki pomiędzy człowiekiem a sztuczną inteligencją

W ramach demonstracji poprawności zaimplementowanych rozwiązań Predictive-Reactive, zarejestrowano wideo dokumentujące starcie pomiędzy graczem a przeciwnikiem. Zamiast obrazów referencyjnych wprowadzono ściśle znaczniki czasowe (minuty:sekundy), pozwalając na podgląd akcji.

- **Inicjalizacja środowiska, nasłuch FSM i patrol [00:00 - 00:05]:** Uruchomienie sceny z widoku edytora i potwierdzenie działania Global Event Bus oraz FSM logami diagnostycznymi w konsoli Unity (m.in. *"Ship stats initialized"*). Zbudowana siatka trójwymiarowych współrzędnych obrazuje przestrzeń operacyjną Voxel Grid algorytmu A*. Ponadto, w początkowej fazie widoczny jest statek przeciwnika realizujący ścieżkę patrolową, podczas której algorytm Minimax wstępnie analizuje przestrzeń pod kątem optymalnego pozycjonowania taktycznego.
- **Analiza fizyczna statku Gracza [00:06 - 00:21]:** Zarejestrowano w panelu *Inspector* bazowe systemy Unity (Rigidbody, Collider), na których nadbudowano systemy balistyki kinetycznej i lotu bezwładnościowego używanego następnie przez A*. Widoczny jest też status wielostrefowego pancerza.
- **Zagęszczenie pola walki i konieczność użycia A* z Raycastami [00:22 - 00:44]:** Ujęcie makro ukazujące olbrzymie pole asteroid udowadnia, że przełot do któregośkolwiek z 7 wylosowanych punktów patrolowych bez połączenia A* i reaktywnego unikania nosowego (3x Raycast) skończyłby się natychmiastową kolizją.
- **Inicjacja kontaktu i system Lead Target [00:45 - 01:05]:** Po wykryciu gracza przez Radar, Event Bus wymusza tranzycję FSM do stanu Combat. AI, sterując wieżyczkami z ograniczeniem kątowym, aplikuje matematykę wyprzedzania strzału (Leading) i posyła czerwone pociski kinetyczne nie w statek gracza, a w docelowy punkt przed nim.
- **Triumf sztucznej inteligencji, błąd ludzki i zniszczenie gracza [01:06 - 01:12]:** Błąd gracza polegający na braku manewrowania został w stu procentach rozczytany przez przewidujący strzał moduł balistyczny Kamila. Obrażenia kierunkowe pancerza strefowego niszczą jednostkę; wyświetla się ekran porażki.
- **Przegrana AI w krytycznym zwarcu [01:13 - 01:37]:** Gracz gwałtownie taranuje przeciwnika i zadaje potężne obrażenia. W sytuacji bez wyjścia i przy spadku HP bota poniżej 25%, Kacper (Mózg AI) próbuje uruchomić stan ratunkowy Flee Mode, ale przewaga ognia niszczy go przed ucieczką. Scena ukazuje poprawne działanie maszyny stanów w sytuacjach bezbłędnej ofensywy gracza i przegraną AI.

- **Dominacja algorytmu Minimax z odcięciem alfa-beta [01:38 - 02:03]:** Praktyczne potwierdzenie optymalności użytego algorytmu. Minimax przetwarza przyszłe ruchy gracza i zamiast kierować przeciwnika prosto na ostrzał frontalny, zmusza statek do szerokiego, prostopadłego manewru burtowego. Maksymalizuje to pole ognia bocznych wieżyczek AI, równocześnie utrudniając graczowi precyzyjne strzały. Zapas HP gracza drastycznie spada [01:52], co kończy się ponowną klęską ludzkiego refleksu w walce ze zoptymalizowaną heurystyką [01:58].
- **Uniwersalność Architektury Standalone (perspektywa bota) [02:04 - 02:53]:** Ujęcie kamery zza statku bota (Third Person). Potwierdza to idealną separację kodu według paradygmatu Standalone. Zbudowany fizyczny silnik lotu, pancerz strefowy i uzbrojenie operują z poziomu Singletona i odbierają komunikaty Event Bus na tyle elastycznie, że te same moduły mogą być pilotowane komendami od AI (A*, Minimax), jak i przez kontrolery gracza bez zmieniania logiki bazowej. Nagranie zamyka ekran zwycięstwa [02:48].

4. Napotkane problemy, wyzwania i sposoby ich rozwiązania

Podczas realizacji zaawansowanej architektury Predictive-Reactive zespół musiał pokonać szereg trudności architektonicznych:

- **Złożoność implementacji zaleconego algorytmu A* w środowisku pełnego 3D:** Przejście z przeszukiwania płaskich siatek 2D na przeszukiwanie objętościowe w kosmosie wymagało sprytnego zoptymalizowania pamięci operacyjnej. **Rozwiązanie:** Skonstruowano od podstaw własną, trójwymiarową przestrzeń wokselową (Voxel Grid) dzielącą sektor na gridy i operującą na niestandardowej euklidesowej funkcji kosztów uwzględniającej gabaryty kadłuba (`ManualAStar.cs`).
- **Spadki wydajności przez obciążenia obliczeniowe algorytmów sztucznej inteligencji:** Równoległe wyznaczanie kilkunastu ścieżek A* w gęstym polu asteroid wraz z jednoczesnym symulowaniem gałęzi decyzji Minimaxa generowało znaczne obciążenie jednostki centralnej (CPU). **Rozwiązanie:** Zredukowano złożoność czasową operacji używając algorytmu odcięcia alfa-beta (dla drzewa Minimax) oraz zaimplementowano A* wewnątrz współprogramów (*Coroutines*), wymuszając asynchroniczny podział operacji na mniejsze zbiory realizowane w kolejnych klatkach obrazu (*time-slicing*). Zapewniło to nienaganną stabilność i płynność działania aplikacji (60 FPS).
- **Dryf bezwładnościowy i problemy celowania (Balistyka):** W środowisku zero-G czysty RigidBody stale uciekał z wektora nadanego mu przez Minimaxa, a pociski nie trafiały ze względu na dodawanie rotacji i pędu. **Rozwiązanie:** Matematycznie odseparowano obrót wieżyczek od samego statku (#185), blokując kąty ostrzału (*Mathf.Clamp*), a dryf kadłuba wygaszano autorskim wektorem stabilizacji bocznej (*antiDriftFactor*).
- **Zbyt wolny czas reakcji A* na bezpośrednie kolizje z asteroidami:** Algorytm wytyczania optymalnej drogi radził sobie z całą siatką nawigacyjną, ale potrafił zignorować szybko mknące fragmenty gruzu z racji bycia za długi na szybki unik na 2 metry przed kolizją. **Rozwiązanie:** Wprowadzono w model *Predictive-Reactive* układ "odruchu bezwarunkowego". Trzy proste promienie nosowe (*Raycasty* #194) w pełni niezależne od A* przejęły odpowiedzialność za fizyczne, błyskawiczne odpychanie maszyny na centymetry przed zderzeniem.
- **Problemy Architektoniczne w Unity:** Klasyczne implementacje i sztywne referencje generowały błędy typu *NullReferenceException* podczas dynamicznego niszczenia i odradzania się wielu obiektów (*respawn*). **Rozwiązanie:** Wdrożono architekturę Standalone opartą w całości na komunikacji pośredniej z wykorzystaniem magistrali zdarzeń (Global Event Bus) oraz wzorca projektowego Singleton (*GameManager.Instance*). Dzięki temu usunięcie przeciwnika wywołuje jedynie zdarzenie w systemie, zwalniając obiekty bez ryzyka wywołania wyjątków w skryptach zależnych.

5. Wykorzystanie Generatywnej Sztucznej Inteligencji

Podczas prac nad projektem wsparto się narzędziami opartymi na generatywnej sztucznej inteligencji (LLM), a dokładnie wykorzystano model **Gemini 3.1 Pro (High)**. Zastosowanie tych rozwiązań miało charakter ściśle pomocniczy i nie obejmowało generowania kluczowej logiki biznesowej gry. Narzędzia AI zostały wykorzystane wyłącznie do dwóch celów:

1. **Redakcja dokumentacji w kodzie:** Narzędzie pomogło zredagować i sformatować utworzone przez nas wcześniej komentarze w kodzie źródłowym. Miało to na celu ujednolicenie stylistyki, poprawienie czytelności oraz nadanie profesjonalnego, inżynierskiego charakteru.
2. **Weryfikacja algorytmów:** Zewnętrzne AI posłużyło jako narzędzie kontrolne do statycznej analizy kodu. Zostało wykorzystane wyłącznie do sprawdzenia i potwierdzenia poprawności matematycznej naszych autorskich implementacji algorytmu decyzyjnego Minimax (wraz z cięciami alfa-beta) oraz algorytmu odnajdywania optymalnej ścieżki A* (A-Star).

6. Podsumowanie i wnioski z implementacji AI

Budowa środowiska opartego wyłącznie na ręcznie zaimplementowanych algorytmach sztucznej inteligencji (A^* dla nawigacji przestrzennej oraz Minimax z odcięciem α - β dla działań taktycznych i walki burtowej), połączona z oddziaływaniem czystej fizyki (Rigidbody), udowadnia, że zaawansowany model obliczeniowy może z powodzeniem sterować modelem statku.

Paradygmat Predictive-Reactive oraz architektura Standalone (GameManager Singleton, Event Bus) wyeliminowały tzw. "sztywne kodowanie", dostarczając dynamiczny ekosystem. Gra reaguje nie tylko na bieżące położenie gracza, ale sprawnie wymusza optymalne formacje obronno-atakujące (manewr burtowy), idealnie radząc sobie z celowaniem wyprzedzającym (Leading). Efektem końcowym jest niepowtarzalny i płynnie działający (60 FPS) symulator bezbłędnej potyczki człowiek-maszyna w środowisku bez grawitacji.